Semana 3 · Cableado, bus I2C dedicado y eco

Fase de la propuesta: puesta en marcha del enlace I2C y calibración del sensor (semanas 3-4).

Objetivo	Cablear el ESP32 a un bus I2C dedicado de la Raspberry, confirmar que el INA219 está en serie con la alimentación y lograr una prueba de eco sin errores a 100 kbit/s.
Dedicación estimada	5–7 horas.
Antes de empezar	Semana 2 cerrada. Resistencias de 4,7 kΩ, cables dupont, protoboard.
Al terminar tendrán	Bus 3 funcionando, ESP32 visible en 0x28 y `docs/evidencias/eco_i2c_f*_*.json` con 0 errores.

> **Seguridad eléctrica**
>
> Hagan todo el cableado con la Raspberry **apagada y desconectada**. El ESP32 y la Raspberry trabajan a **3,3 V** en sus GPIO: nunca conecten 5 V a un pin de señal. Las resistencias pull-up van a **3,3 V** (pin 17), no a 5 V.

Paso 1. Confirmar que el INA219 mide la corriente de la Raspberry

Para medir el consumo, el INA219 debe estar **en serie** con los 5 V que alimentan la Raspberry: fuente (+) → VIN+ ; VIN− → 5 V de la Raspberry. Las líneas SDA/SCL del sensor van al bus 1 (pines 3 y 5).

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 potencia.py --salida /tmp/reposo.csv --duracion 15
python3 potencia.py --resumen /tmp/reposo.csv
stress-ng --cpu 4 --timeout 25s &
python3 potencia.py --salida /tmp/carga.csv --duracion 15; wait
python3 potencia.py --resumen /tmp/carga.csv
```

- **Correcto:** en reposo la corriente es de varios cientos de mA (una Raspberry 4 en reposo consume del orden de 2,5–3,5 W) y con `stress-ng` sube claramente.
- **Incorrecto:** corriente cercana a 0 o que no cambia con la carga: el sensor está en paralelo o mide otra cosa. Corrijan el cableado antes de seguir.

> **Cómo pasar los 5 V por el INA219**
>
> Lo más seguro es un adaptador o cable USB-C con acceso al conductor de +5 V (se corta solo ese conductor y se intercala el INA219). Alimentar por los pines 5 V del GPIO también funciona, pero salta el fusible de protección de la placa: usen la fuente oficial y revisen la polaridad dos veces.

Paso 2. Habilitar el bus 3 para el enlace y subir el bus 1 a 400 kbit/s

```
sudo nano /boot/firmware/config.txt
#   agreguen al final (o ajusten si ya existen):
dtparam=i2c_arm=on
dtparam=i2c_arm_baudrate=400000
dtoverlay=i2c3,pins_4_5,baudrate=100000
#   guardar: Ctrl+O, Enter ; salir: Ctrl+X
sudo reboot
```

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
ls /dev/i2c-*                  # deben existir i2c-1 e i2c-3
i2cdetect -y 1                 # INA219 en 40
python3 sistema.py             # 'i2c_hz': 100000 (bus 3)
```

Paso 3. Cablear el ESP32

Señal	ESP32 DevKit clásico	Raspberry Pi 4 (GPIO / pin físico)	Nota
SDA del enlace	GPIO21	GPIO4 / pin 7	pull-up de 4,7 kΩ a 3,3 V
SCL del enlace	GPIO22	GPIO5 / pin 29	pull-up de 4,7 kΩ a 3,3 V
READY	GPIO4	GPIO17 / pin 11	salida del ESP32, entrada de la Pi
GND	GND	GND / pin 9	referencia común (obligatorio)
3,3 V para pull-ups	—	3V3 / pin 17	solo alimenta las dos resistencias
Alimentación del ESP32	USB	un puerto USB de la Pi	así su consumo entra en la medición
INA219 (bus 1)	—	VCC pin 1, GND pin 6, SDA pin 3, SCL pin 5	ya conectado

> **Si su ESP32 no es el DevKit clásico**
>
> Elijan dos GPIO libres para SDA/SCL y uno para READY, **evitando los pines de arranque** del modelo (consulten su hoja de datos), y cambien `PIN_SDA`, `PIN_SCL` y `PIN_READY` al inicio de `esp32/firmware/main/main.c`.

Paso 4. Grabar el firmware de eco (tramas de 32 bytes)

En la laptop, editen `esp32/firmware/main/CMakeLists.txt`:

```
set(FRAME_SIZE 32)
set(MODO_ECO 1)
```

```
cd ~/esp/proyecto/esp32/firmware
idf.py -p PUERTO build flash
```

Pasen el cable USB del ESP32 de la laptop a un puerto USB de la Raspberry y abran la consola serie desde la Pi:

```
python3 -m serial.tools.miniterm /dev/ttyUSB0 115200     # salir: Ctrl+]
#   debe mostrar: MODO ECO listo. Direccion 0x28, trama 32 bytes
i2cdetect -y 3                                              # debe aparecer 28
```

Paso 5. Prueba de eco

La Raspberry escribe una trama aleatoria y la lee de vuelta; si llega idéntica, el bus transmitió sin errores. Primero 1.000 tramas y luego 10.000:

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 prueba_eco.py --frame 32 --n 1000
python3 prueba_eco.py --frame 32 --n 10000
```

Repitan con tramas de **64** y **128** bytes (cambien `FRAME_SIZE` en el CMakeLists, vuelvan a grabar y usen el mismo `--frame`). Cada prueba deja un JSON en `docs/evidencias/`.

Campo del JSON	Qué significa	Esperado
correctas / distintas	tramas devueltas idénticas / con algún bit cambiado	distintas = 0
errores_io	la Raspberry no pudo escribir o leer (NACK, sin respuesta)	0
timeouts_ready	el ESP32 no levantó READY a tiempo	0 (o muy pocos)
tasa_bus_kbps	tasa efectiva del bus (incluye esperas)	< 100 a 100 kbit/s
i2c_hz	frecuencia configurada del bus 3	100000

Si falla

- **errores_io altos** (Remote I/O error): revisen GND común, SDA/SCL invertidos, pull-ups, dirección 0x28 y que el firmware sea el de eco.
- **timeouts_ready altos**: cable READY suelto o en otro pin; revisen GPIO4 (ESP32) → GPIO17 (Pi).
- **distintas > 0**: es el problema de *clock stretching* de la Raspberry. Prueben tramas de 32 B; si persiste, cambien la línea del bus 3 por un bus I2C por software (maneja bien el clock stretching):

```
dtoverlay=i2c-gpio,bus=3,i2c_gpio_sda=4,i2c_gpio_scl=5,i2c_gpio_delay_us=2
```

Anoten en `docs/decisiones.md` cualquier cambio de este tipo: es un hallazgo para la tesis.

Criterio de cierre de la semana

- ☐ El INA219 mide la corriente de la Raspberry (sube con stress-ng).
- ☐ /dev/i2c-3 existe a 100 kHz y el ESP32 responde en 0x28.
- ☐ Prueba de eco de 10.000 tramas con 0 distintas y 0 errores_io para al menos un tamaño de trama.
- ☐ Resultados de 32, 64 y 128 bytes guardados en docs/evidencias/.
- ☐ Bitácora con una foto del cableado; commit y push.